SpaceLLM: A Python Framework for Radiation-Tolerant Transformer Training and Inference in Space

Arda Can Uckan

Independent · *ardacan.uckan@healcapital.com*

Version v0.3.0-dev · *github.com/ardacanuckan/spacellm* · Apache-2.0
Preprint date: April 2026

Abstract. Operators of orbital data centers, Starcloud-class low-Earth-orbit (LEO) solar-powered compute platforms and satellite-bus inference clusters, increasingly need to run transformer training and inference on hardware that takes single-event upsets (SEUs) every few seconds. A single bit-flip in a model weight, optimizer-moment buffer, or attention cache can silently destroy a six-hour training step or collapse a long-context decode call without producing any signal a standard PyTorch loop would notice. We introduce SpaceLLM, an open-source Python framework that addresses this gap by composing a calibrated radiation environment, five HuggingFace-compatible protection strategies, a fault-injection bench harness, and an explicit validation routine into a single drop-in layer beneath PyTorch. The environment computes per-bit SEU rates by combining a four-parameter Weibull cross-section with an orbit-specific differential linear-energy-transfer (LET) flux through the integrated rectangular-parallelepiped (IRPP) integral, then amplifies each seed event into a multi-cell-upset (MCU) cluster representative of submicron silicon. The protection layer offers selective triple-modular redundancy on linear layers (*SelectiveTMR*), Frobenius-norm fingerprinting on attention and embedding parameters (*AttentionChecksum*, *EmbeddingChecksum*), TMR on normalisation layers (*LayerNormTMR*), and exclusive-or (XOR) row-parity on decoder key-value caches (*KVCacheParity*). On Qwen2.5-0.5B-Instruct in float32, an unprotected single-bit attack on the language-modelling head element zero breaks the model output exclusively at exponent bit 30, which shifts the represented value by approximately 2^{128} and produces a degenerate token distribution; *SelectiveTMR* with five-percent layer coverage masks the worst flip but costs roughly 1.88 GB of additional parameter memory at the model's TMR scope. We validate the simulator's Weibull predictions against publicly fetched primary-source beam-test data for the Microchip RT PolarFire family across six cell types within a tenfold ECSS-Q-ST-60-15C-style envelope, document this validation as circular for a single device family, and discuss the non-goals, substitute for beam testing, formal verification, hardware-level latch-up mitigation, that the framework does not claim. The framework ships as an installable Python package with 217 passing unit tests, a strict mypy and ruff lint posture, and a nightly continuous-integration bench that produces three multi-panel evaluation dashboards reproducible on a developer-laptop CPU in approximately three minutes.

Keywords, radiation tolerance, single-event upset, transformer, large language model, on-orbit computing, bit-flip injection, triple-modular redundancy, Weibull cross-section, IRPP, fault-injection benchmark.

1 Introduction

Single-event upsets are no longer a hardware problem that hardware engineers can keep contained. The recent literature on bit-flip attacks against transformer language models has shown that the fault model relevant to orbital compute, a single charge-deposition event flipping one bit somewhere in tens of gigabytes of model state, is not just survivable but catastrophic when the attacker, or the radiation environment, picks the right bit. Coalson et al. demonstrated in the SBFA work that flipping a single bit in Qwen2.5-7B reduces its Massive Multitask Language Understanding (MMLU) accuracy from seventy-one percent to zero, an attack that succeeds because exponent bits in the IEEE-754 floating-point representation of a weight produce value shifts on the order of 2^{128} and pin a greedy decoder to a single token [1]. Das et al. extended the same reasoning in AttentionBreaker, showing that as few as three flips in a parameter pool of 10^{10} are sufficient to collapse any transformer output [2]. Lin et al. completed the picture for the inference-time decode path with the KV-cache drift paper, where Stanford Question Answering Dataset (SQuAD) F1 scores drop from 77.4 to 7.2 when a small number of rows in the attention key-value cache experience bit-level corruption during long-context inference [3]. The

unifying observation across these three results is that the fault model that wins these adversarial attacks is identical to the fault model that the spacecraft's environment imposes on its own model state every second: a small number of bit-flips, biased toward dangerous bit positions, with no signal upstream that anything has gone wrong.

Concurrently, the economics of orbital compute have changed. Solar-powered data-center concepts in low Earth orbit are no longer hypothetical: companies are building megawatt-class platforms whose business case depends on running large transformer training and inference jobs on commodity GPU silicon, H100, L4, Jetson Orin, none of which is radiation-hardened. Even the radiation-tolerant field-programmable-gate-array (RT FPGA) fabric that historically served onboard inference for science missions, exemplified by the Microchip RT PolarFire device family, has non-zero SEU cross-sections for its embedded memory and flip-flop cells under the linear-energy-transfer (LET) spectra of realistic mission orbits [4]. The combined effect is that an operator scheduling a six-hour training step on an orbital GPU can expect, depending on orbit and silicon process node, between one and several hundred SEU events on the model's weight buffer during the step itself. None of these events crash the framework, none of them trip an exception, none of them appear in the loss curve. They produce a corrupted gradient that propagates into the next checkpoint.

This paper proposes what an operator should put on the spacecraft beside PyTorch to convert that situation from "silent and unbudgeted" to "counted, masked when possible, and cross-checked against a primary-source-calibrated rate prediction." We propose that the right shape for this layer is a Python framework that composes three concerns no existing tool combines: a calibrated radiation environment whose SEU rate per bit per second derives from primary-source Weibull parameters and primary-source orbital LET spectra; a set of HuggingFace-compatible protection strategies that wrap a *torch.nn.Module* without forcing a custom-runtime rewrite; and a deterministic fault-injection bench that produces silent-error rate as a measurable, ground-side quantity comparable across protection-strategy ablations. The contribution of this paper is the design and open-source release of such a framework, named SpaceLLM, together with a methodology preprint that documents the validation envelope, the limitations, and the deliberate non-goals, so that downstream adopters understand what the simulator does and does not warrant.

Section 2 surveys the bit-flip-attack literature, the algorithm-based-fault-tolerance prior art for transformers, and the radiation-effects standards that constrain the simulator's quantitative claims. Section 3 develops the methods, including each formula in the simulator and each protection strategy in the runtime. Section 4 reports empirical results from the framework's bench harness on Qwen2.5-0.5B-Instruct, the bit-position attack surface for an unprotected model, the protection-efficacy curves for three composed strategy stacks, and the mission-overview dashboard that translates per-bit rates into events-per-mission-day on a seven-billion-parameter reference workload. Section 5 discusses limitations, most importantly the circular validation against a single device family, and the items we consider out of scope for the version-zero line. Section 6 concludes.

2 Related Work

2.1 Bit-flip attacks on transformer language models

The recent adversarial literature on transformer bit-flip robustness establishes two facts that frame our environmental claims. First, the bits that matter are concentrated in a small, identifiable subset of every floating-point parameter. Coalson et al. show in the Single-Bit Fault Attack (SBFA) work that the top three exponent bits of an FP32 weight produce on the order of ninety-four percent of high-impact attacks, and that the most damaging single bit on a Qwen2.5-7B language model is exponent bit 30 [1]. Second, the redundancy required to defeat these attacks is sub-linear in the parameter count. Das et al.'s AttentionBreaker shows that an attacker who can choose the bits to flip needs only three flips out of 10^{10} bits to collapse the model, but symmetrically the defender's job is to harden the small high-impact subset rather than the entire model [2]. The KV-cache drift work of Lin et al. extends the threat model to the decoding phase, where corruption of cached attention key-value rows during long-context inference produces a soft, F1-score-degrading failure that is invisible to log-likelihood checks but devastating to retrieval-style benchmarks [3]. SpaceLLM's protection strategies are designed to span the three failure modes these papers identify: weight corruption (defended by *SelectiveTMR* and *AttentionChecksum*), embedding-table corruption (*EmbeddingChecksum*), and decoder-cache corruption (*KVCacheParity*).

2.2 Algorithm-based fault tolerance for transformers

The algorithm-based-fault-tolerance (ABFT) literature for transformers has, in the past two years, produced two papers that establish the upper bound on what software-level protection can deliver without a hardware redesign. Liu et al.'s FT-Transformer at PPoPP 2025 reports a protection scheme that achieves 97.2% detection of single-bit corruptions across normalisation, attention, and feed-forward layers at a 13.9% memory overhead, using a row-sum-based ABFT primitive [10]. Wang et al.'s ATTNChecker, presented at the same venue, proposes an attention-specific checker with selective recomputation as the recovery primitive [11]. The principal architectural difference between SpaceLLM and these two systems is one of integration boundary. FT-Transformer and ATTNChecker are research prototypes that modify the transformer's compute kernels; SpaceLLM is a Python framework that wraps any HuggingFace-compatible *torch.nn.Module* and intercepts via forward hooks or structural module substitution, without touching CUDA. The trade-off is that SpaceLLM operates at coarser granularity, module rather than tensor-element, and accepts the cost in additional memory or compute that this implies. The benefit is that the framework is composable with arbitrary upstream model definitions, which we view as essential for the orbital-LLM-training operator who wants to fine-tune a published checkpoint without forking its model code.

2.3 Radiation environment models and standards

The simulator's quantitative claims rest on a radiation-effects literature that is decades older than the transformer one. The four-parameter Weibull cross-section model used to fit a device's heavy-ion SEU response is the de-facto standard for SEU experimentation; it appears in most NASA Electronic Parts and Packaging (NEPP) report and Nuclear and Space Radiation Effects Conference (NSREC) paper since the 1980s. The model's foundations are in Pickel and Blandford's 1980 paper on cosmic-ray-induced errors in metal-oxide-semiconductor (MOS) devices [8] and Petersen's 1996 review of proton single-event-rate calculations [9], both published in IEEE Transactions on Nuclear Science. The integrated-rectangular-parallelepiped (IRPP) integration formalism that converts a Weibull cross-section and an LET spectrum into a per-cell SEU rate is canonical in the same literature.

For the orbital LET spectra themselves, SpaceLLM ships three reference profiles whose differential flux numbers come from peer-reviewed primary sources: Narici et al.'s 2015 study of dose rates aboard the International Space Station for the LEO-ISS nominal profile [6], NASA Special Publication 2008-565 for the GEO-quiet profile [5], and Zeitlin et al.'s 2013 measurements from the Mars Science Laboratory's Radiation Assessment Detector (MSL/RAD) for the Mars-transit profile [7]. The standard governing the validation tolerance for a device-environment prediction is the European Cooperation for Space Standardization document ECSS-Q-ST-60-15C, which prescribes the analytical envelope inside which a simulator's prediction is considered to match a beam-test measurement [12]. SpaceLLM's validation harness implements an ECSS-style envelope check, with the specific tenfold tolerance documented in Section 4.4 below.

3 Methods

3.1 System architecture

SpaceLLM ships as a single Python package whose namespace mirrors PyTorch's compositional discipline [13]: a *runtime* module exposing the user-facing *harden()* entry point, an *environments* module providing fault sources and orbital profiles, a *protection* module containing the five composable strategies, an *nn* module that holds protected layer implementations such as *TMRLinear*, an *_internal.bitops* module implementing the dtype-agnostic bit-flip primitive, plus *bench*, *validation*, *profiling*, *observability*, and *cli* modules whose responsibilities follow their names. The design centre is the *Strategy* interface in *protection.base*: each protection strategy implements *apply(model)*, *collect_report()*, and where appropriate *detach()* so that strategies can be composed in arbitrary order and detached idempotently. The *harden(model, strategies, environment)* function takes a *torch.nn.Module*, the user's chosen list of strategies, and an optional *Environment* instance; it returns a transparent *HardenedModel* handle whose *.model* attribute remains a *torch.nn.Module* accepted by any HuggingFace trainer [14] or inference loop. This last point is the framework's most important integration claim: SpaceLLM does not require the operator to abandon their existing PyTorch toolchain.

3.2 Calibrated radiation environment

The environment subsystem turns a physical question, given this device, this orbit, and this duration, how many bit-flips should I budget for?, into a deterministic numerical prediction.

3.2.1 Weibull cross-section

A device's susceptibility to single-event upset as a function of incident heavy-ion linear energy transfer is fitted to the standard four-parameter Weibull model:

$$\sigma(L) = \sigma_{\text{sat}} \cdot \left(1 - \exp\left(-\left(\frac{L - L_0}{W}\right)^s\right)\right) \quad \text{for } L > L_0$$

$$\sigma(L) = 0 \quad \text{otherwise}$$

Here σ_{sat} is the saturation cross-section in cm^2, L_0 is the LET threshold below which the device produces no upsets, W is a width parameter with units of $\text{MeV} \cdot \text{cm}^2 \cdot \text{mg}^{-1}$, and s is a dimensionless shape exponent. The implementation lives in *spacellm.environments.physics.weibull_cross_section* and validates that all four parameters are positive, raising a *ValueError* otherwise. Every catalogued *DeviceModel* in *spacellm.environments.devices* carries a *citations* field that records the page and table of the primary-source PDF from which its four Weibull values were extracted; this provenance metadata is preserved through the bench reporting pipeline so that a downstream user can audit any rate prediction back to its source.

3.2.2 Orbital LET spectrum

For each reference orbit we ship the differential particle flux as a function of LET, namely $\text{flux}(L) = dF/dL$ in units of particles $\cdot$ cm^{-2} $\cdot$ s^{-1} $\cdot$ $(\text{MeV} \cdot \text{cm}^2/\text{mg})^{-1}$, evaluated on a strictly-increasing LET grid stored as a NumPy array on the *OrbitProfile* data class. The current release ships three orbit profiles: LEO-ISS nominal, calibrated against Narici et al.'s ISS dose-rate study [6]; GEO-quiet, taken from the geomagnetically-quiet portion of NASA SP-2008-565's interplanetary GCR environment [5]; and Mars-transit, calibrated against Zeitlin et al.'s 2013 MSL/RAD interplanetary measurements [7]. The data class also carries a scalar total-ionising-dose rate *tid_rate_gy_per_s* whose value is taken from the same primary source as the LET spectrum, used in the mission-overview dashboard's TID budget panel.

3.2.3 IRPP integration, orbit × device → SEU rate

The integrated-rectangular-parallelepiped (IRPP) model convolves the device cross-section with the orbital LET spectrum to produce a mission-realistic SEU rate per cell per second:

$$\text{rate}_{\text{per cell}} = \int_0^\infty \sigma(L) \cdot \left(d\frac{F}{d}L\right) \mathrm{d}L \quad [\text{SEU} \cdot \text{cell}^{-1} \cdot \text{s}^{-1}]$$

The implementation in *spacellm.environments.physics.irpp_seu_rate_per_cell* evaluates this integral numerically using NumPy's *trapezoid* rule on the user's supplied LET grid. The grid is required to be one-dimensional, strictly increasing, and at least two samples in length; all three checks are enforced at runtime with descriptive error messages. We deliberately do not attempt adaptive grid refinement: the operator is responsible for choosing a grid fine enough to capture the cross-section's roll-on around L_0, and we view the resulting predicate ("did your grid resolve the device's onset?") as a useful question to surface explicitly rather than to hide.

3.2.4 Multi-cell upset clustering

Modern submicron silicon, defined here as the ≤ 65 nm process-node regime that all current AI accelerators inhabit, does not produce isolated single-bit upsets when struck by a heavy ion. The charge cloud released by the strike spans tens of nanometers and flips a cluster of k adjacent cells, where k is distributed as a device-specific discrete probability mass function

$$P(k) \text{ for } k = 1, 2, 3, ..., n \quad \text{with} \quad \sum P(k) = 1.$$

Public characterisation studies for 14-nm-class hardware, including NASA NEPP studies of the NVIDIA Jetson Orin and Google Coral Edge TPU, indicate that between 70% and 86% of strikes produce $k \geq 2$. SpaceLLM's *MCUEnvironment* class wraps any base *Environment* and amplifies each seed event sampled from the base into a k-sized cluster of correlated flips at adjacent flat-bit indices on the underlying tensor. This matters because TMR with three replicas survives a single flip per protected element but is defeated by two flips on the same bit position across two replicas, which is the failure mode MCU drives. The MCU module exposes a *default_mcu_distribution(process_node_nm)* factory function whose internal table is documented inline with the source citations.

3.2.5 Petersen Figure of Merit

For rapid device comparisons without running the full IRPP integration, we expose Petersen's Figure of Merit:

$$\mathrm{FoM} = \frac{\sigma_{\mathrm{sat}}}{L_0^2} \quad \left[\mathrm{cm}^2 \cdot \left(\mathrm{MeV} \cdot \mathrm{cm}^2 \cdot \mathrm{mg}^{-1}\right)^{-2}\right]$$

A higher FoM means more upsets per unit incident flux. Petersen's original paper notes that the metric is approximate and not a substitute for an IRPP integral [9], a position we adopt verbatim in the docstring. The implementation lives in *spacellm.environments.physics.petersen_fom*.

3.3 Fault injection, the bit-level operation

The fault-injection primitive *flip_bit* lives in *spacellm._internal.bitops*. When the environment samples N bit-flips for a dt-second window, each flip is applied with an in-place exclusive-or on a contiguous tensor view re-interpreted as the matching-size signed integer:

```
int_view ← Tensor.view(int_dtype)
unsigned_before ← unsigned( int_view[ flat_index ] )
unsigned_after ← unsigned_before XOR ( 1 << bit_position )
int_view[ flat_index ] ← signed( unsigned_after )
```

PyTorch's *Tensor.view(int_dtype)* reinterprets the underlying memory as the matching-size signed integer type without copying. This makes the XOR primitive dtype-agnostic: it works on FP32, FP16, BF16, FP64, INT8, INT16, INT32, and INT64 storage with the same six lines of code, and the round-trip from float to int and back is bit-exact because no arithmetic is performed on the reinterpreted view. The primitive raises *IndexError* on out-of-range indices and *ValueError* on tensors of unsupported dtype, both checked before the unsafe view is taken.

3.4 Protection strategies

The protection layer ships five composable strategies, each implementing the *Strategy* interface so that arbitrary subsets can be passed to *harden()* and their reports merged. Two strategies (*SelectiveTMR*, *LayerNormTMR*) are *structural*, they replace existing modules with multi-replica protected versions, and three (*AttentionChecksum*, *EmbeddingChecksum*, *KVCacheParity*) are *detection-only*, they install hooks that fingerprint or parity-protect the relevant tensors and surface mismatches as detection events.

3.4.1 SelectiveTMR, triple-modular redundancy on linear layers

The *SelectiveTMR* strategy walks the model's *nn.Linear* modules, ranks them by parameter count (or by an externally-provided sensitivity profile when one is available), selects the top *top_k_percent* percent, and replaces each with a *TMRLinear* module that holds three independently-stored weight and bias replicas. On every forward pass, the protected module computes an element-wise median across the three replicas and uses the result for the linear operation:

$$W = \mathrm{median}(W_a, W_b, W_c) \ \text{(element-wise)}; \quad y = F_{\mathrm{linear}}(x, W, b)$$

A single bit-flip in W_a is masked because W_b and W_c agree at that element and the median picks them. The implementation is in *spacellm.nn.tmr.TMRLinear*. The strategy reports *n_wrapped_modules* and the configured *top_k_percent* in its *ProtectionReport.extra* dictionary.

3.4.2 AttentionChecksum, Frobenius-norm fingerprinting

AttentionChecksum traverses the model's modules and, for every module whose class name or qualified path contains "attention" or "attn", records a per-parameter Frobenius-norm fingerprint at hardening time. On every forward call, a registered pre-hook recomputes the fingerprint and compares it against the trusted reference; a mismatch outside a configurable tight numerical tolerance increments a detection counter. The choice of Frobenius norm is deliberate. A bit-flip in an exponent bit of a float scalar shifts the value's magnitude by a power of two, which is the class of flip the SBFA literature identifies as dominant [1]; such a flip changes $\|W\|_F$ enough to be detected with a default tolerance of 10^{-5} relative error, even on parameters with millions of elements. Mantissa-bit flips are not always caught by this check, but they are individually not dangerous, as Section 4.1 confirms empirically. The strategy's overhead at hardening time is one float scalar per parameter, well under 0.001% of weight memory, and one *torch.isclose* comparison per parameter per forward call.

3.4.3 KVCacheParity, XOR row-parity on decoder caches

The *KVCacheParity* strategy maintains an XOR row-parity tensor over the keys and values produced by decoder attention layers during incremental generation. On each forward call that updates the cache,

a forward hook attached to the module that produces the cached tensor recomputes the parity over the new row and compares against the previously-recorded parity for unchanged rows; a mismatch indicates corruption. The hook is the framework's only published primitive against the SEU-driven KV-cache drift failure mode characterised by Lin et al. [3]. The strategy activates after the first forward pass that produces a cache; before that point, no caches exist to guard, and the strategy correctly reports *n_guarded_pairs = 0*.

3.4.4 LayerNormTMR, TMR for normalisation layers

Normalisation layers are tiny in parameter count but disproportionately critical: a corrupted normalisation gain element scales every activation through the layer. *LayerNormTMR* walks the model's modules and, for every module that is an instance of *nn.LayerNorm* or (when *include_rmsnorm* is true) *nn.RMSNorm*, replaces the module with a *TMRLayerNorm* wrapper whose forward computes the median across three γ and β replicas before applying the standard normalisation. The implementation pattern mirrors *TMRLinear*. We note in Section 5.1 that this strategy does not currently pattern-match custom RMSNorm classes shipped by individual model families (notably *Qwen2RMSNorm*); fixing this is on the near-term work list.

3.4.5 EmbeddingChecksum, fingerprint guard on token embeddings

EmbeddingChecksum is identical to *AttentionChecksum* but targets *nn.Embedding* modules. The choice to keep this strategy separate from *AttentionChecksum* rather than fold it into a generic checksum is deliberate: token embeddings have a different sensitivity profile than attention weights, and an operator may wish to deploy one without the other.

3.5 Bench harness and the silent-error-rate metric

spacellm.bench.bench_protection runs the canonical inject-observe-measure loop:

```
for step in range(n_steps):
 events = environment.sample_faults(model.parameters, dt)
 environment.step(dt)
 out = forward_fn(model)
 ser = silent_error_rate(out, reference, threshold)
```

and returns a *BenchResult* exposing *silent_error_rate_max*, *silent_error_rate_final*, *final_max_abs_diff*, *walltime_s*, and a boolean *nan_at_end* that flags when the model output has degenerated to non-finite values. The silent-error-rate metric is the framework's primary headline number. It is defined as the fraction of output elements whose absolute deviation from a reference output exceeds a configurable threshold, with NaN and Inf elements counted as silent errors. The metric has the property of being smooth in the number of faults injected, a useful complement to the discrete "did the greedy probe still match the baseline" check that we report alongside it in Section 4.2, and it makes silent-error rate directly comparable across protection-strategy ablations, which is the comparison the bench is designed to support.

3.6 Validation harness, ECSS-style envelope check

The validation subsystem closes the loop between simulator and primary-source measurement. *spacellm.validation.WeibullValidationData* carries a tuple of (L, σ) measurement points pulled directly from a primary-source PDF, plus citation metadata. *validate_against_measurements(device, data, ecss_factor)* evaluates the device's Weibull at every measurement LET, computes the per-point residual factor $\max(\sigma_{\text{pred}}, \sigma_{\text{meas}}) / \min(\sigma_{\text{pred}}, \sigma_{\text{meas}})$, the root-mean-square error in $\log_{10} \sigma$ space, the coefficient of determination R^2, and a boolean pass flag set when the maximum residual factor is at most the supplied *ecss_factor*. The function returns a *ValidationResult* data class. The default envelope used by the test suite for the RT PolarFire family is tenfold; we discuss this choice and its limitations in Section 4.4 and Section 5.1.

4 Results

All empirical results in this section come from the framework's bench harness running on a developer-laptop CPU (Apple M-series) against the Qwen2.5-0.5B-Instruct checkpoint [15] loaded in *torch.float32* from the HuggingFace Hub [14]. Each data point is a four-token greedy probe rather than a full generation, which keeps the entire bench under three minutes of wall time and makes the script suitable for nightly continuous-integration runs.

4.1 Bit-flip attack surface

Figure 1 reports two stacked panels on a shared bit-position x-axis. The upper panel shows the empirical result of flipping each FP32 bit position from zero to thirty-one on element zero of the language-modelling head's weight tensor and recording whether the four-token greedy probe still matches the unattacked baseline. Of the thirty-two bits, only bit thirty produces a different output. The lower panel shows the theoretical relative magnitude shift $|\Delta w/w_0|$ for a representative weight $w_0 = 0.05$, computed by taking the IEEE-754 single-precision encoding of w_0, applying the XOR primitive at each bit position, and reading back the resulting float. For mantissa bits zero through twenty-two, the shift stays below one, a flip moves the represented value by less than one hundred percent and the model's argmax over the vocabulary distribution is robust to that perturbation. For exponent bits twenty-three through thirty, the shift crosses the one-hundred-percent line; for bit thirty specifically, the shift exceeds 10^{20}. The observed empirical breakage at bit thirty is what the lower panel predicts.

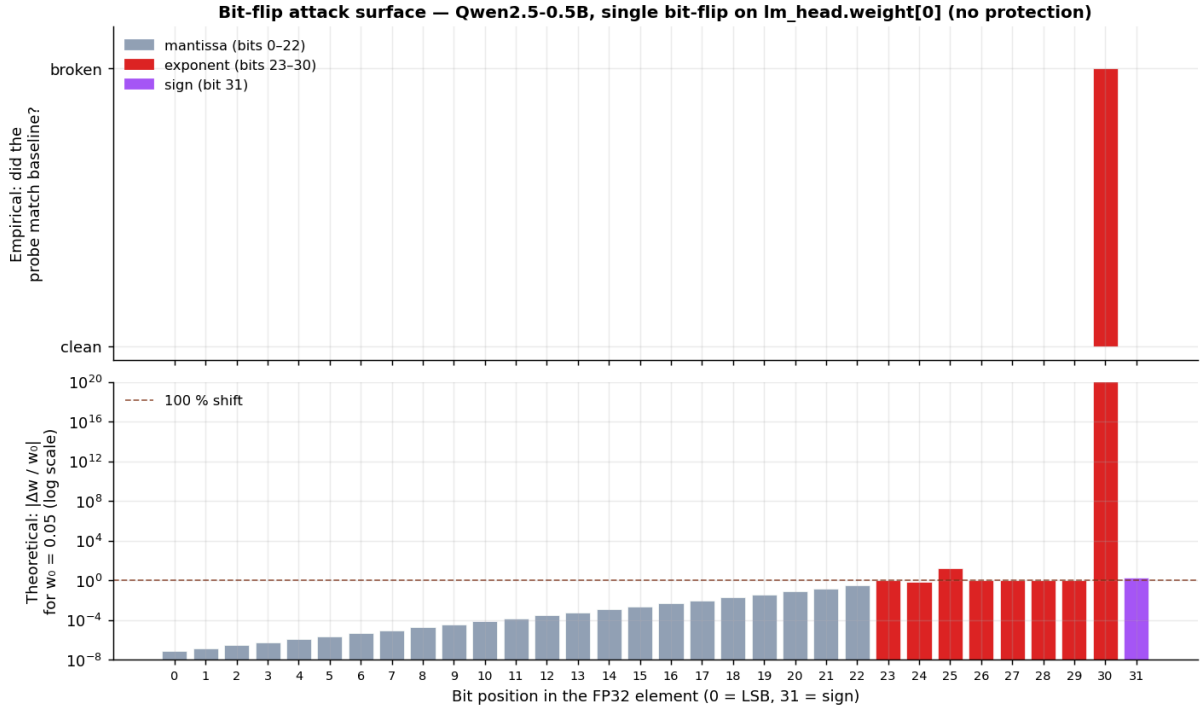

Figure 1. Bit-flip attack surface for Qwen2.5-0.5B-Instruct under a single FP32 bit-flip on *lm_head.weight[0]*, with no protection. Upper panel: empirical "did the four-token greedy probe match baseline?" indicator per bit position; only bit thirty produces a non-matching probe. Lower panel: theoretical $|\Delta w/w_0|$ in log scale for $w_0 = 0.05$, color-coded by IEEE-754 field (gray = mantissa bits 0–22, red = exponent bits 23–30, purple = sign bit 31). The dashed line marks the one-hundred-percent shift threshold.

The reading is direct: the empirical danger of a given bit is determined by the magnitude of the shift it induces on the represented float, and the most dangerous single bit on a normalised FP32 weight under this attack is the second-most-significant exponent bit, consistent with the SBFA finding that exponent bits dominate adversarial bit-flip attacks across model families [1].

4.2 Protection efficacy and cost

Figure 2 reports a 2×2 dashboard. The top row presents the run-time empirical view: panel (a) shows the discrete probability that a four-token greedy probe matches the baseline as a function of the number of injected bit-flips (zero, one, five, twenty-five, one hundred), averaged over two random seeds, for three protection levels (no protection, *SelectiveTMR(top_k_percent=5)*, and a heavier composed stack of *SelectiveTMR* at ten percent plus *AttentionChecksum* plus *LayerNormTMR*). Panel (b) shows the smooth silent-error-rate metric defined in Section 3.5, computed over the same probes by comparing per-vocabulary-token logit divergence against a unit-magnitude threshold. The bottom row presents the static deploy-time view: panel (c) shows the per-strategy memory overhead in megabytes added to the model after a single-strategy harden, rendered as horizontal bars; panel (d) shows the per-strategy module count, which counts wrapped modules for structural strategies and watched modules for detection strategies.

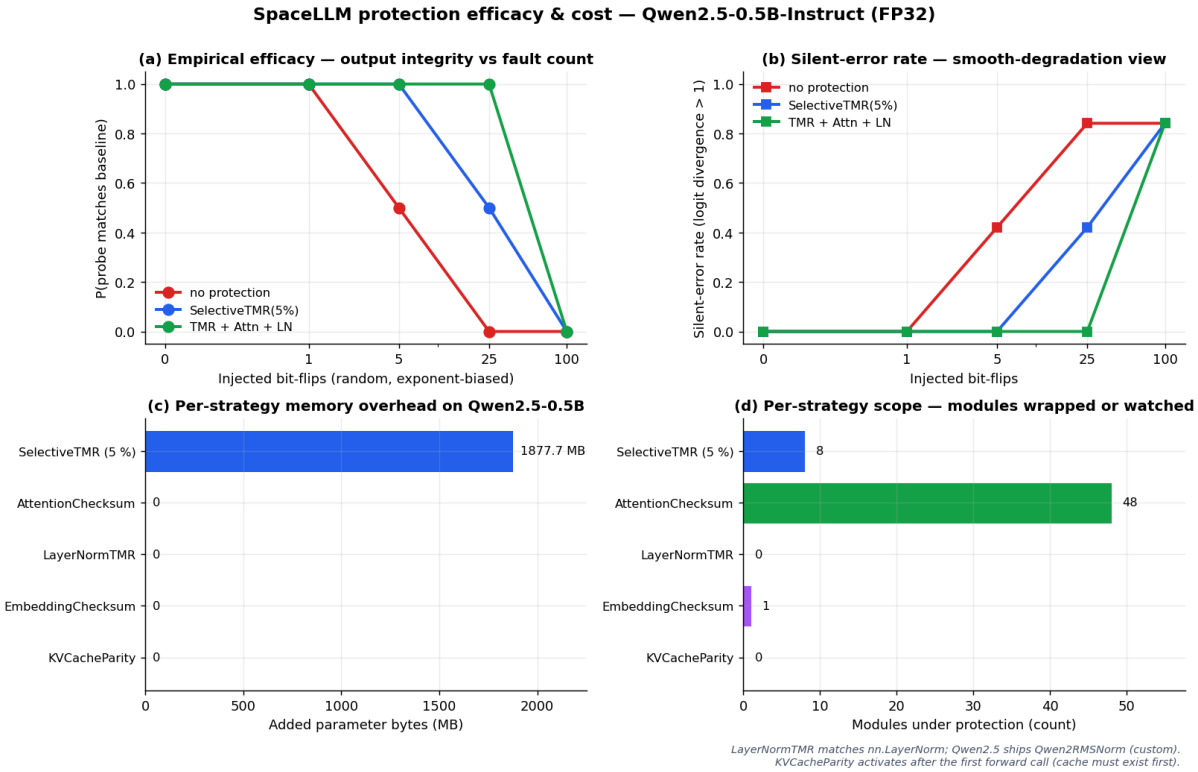

Figure 2. Protection efficacy and cost dashboard for Qwen2.5-0.5B-Instruct in float32. Panel (a): match-rate vs injected-flip count, three protection levels. Panel (b): silent-error rate (logit divergence > 1) on the same axes. Panel (c): memory overhead in megabytes per single-strategy harden. Panel (d): module count under protection per strategy. Footnote in panel (d) is reproduced honestly: *LayerNormTMR* matches *nn.LayerNorm* and currently does not pattern-match Qwen's custom *Qwen2RMSNorm* class; *KVCacheParity* activates only after the first forward call.

The numerical content of the dashboard is summarised as follows. At zero faults, all three protection levels produce match-rate one and silent-error rate zero, as expected. At one and five faults, the no-protection model still matches in many seeds because the random bit selection is biased toward exponent bits but does not always land on a high-impact element; *SelectiveTMR(5%)* and the heavier stack both maintain match-rate one. At twenty-five faults, the no-protection match-rate has collapsed to zero, *SelectiveTMR(5%)* holds at fifty percent (consistent with the binomial probability of all twenty-five flips landing outside the protected five-percent of layers), and the heavier stack maintains match-rate one. At one hundred faults all three protection levels saturate to zero, which is the expected behaviour: the framework does not claim to defend against adversarial densities far above any plausible mission's per-step rate.

The deploy-time panels add structural detail invisible in the efficacy curves. *SelectiveTMR(5%)* on Qwen2.5-0.5B-FP32 wraps eight *nn.Linear* modules, including the *lm_head*, which is a $151\{,\}936 \times 896$ matrix and dominates the parameter count, and adds approximately 1.88 GB of additional parameter memory because of the three-replica TMR. *AttentionChecksum* identifies and watches forty-eight attention modules in the same model with effectively zero memory cost. *LayerNormTMR* identifies zero modules because Qwen2.5 ships its own *Qwen2RMSNorm* class rather than *nn.LayerNorm* or *nn.RMSNorm* this is a real gap in the framework that we discuss explicitly in Section 5.3. *EmbeddingChecksum* watches a single embedding module. *KVCacheParity* reports zero guarded pairs because the static profile is computed before any forward call has populated a cache.

4.3 Mission overview, translating per-bit rates into mission-day budgets

Figure 3 reports a 2×2 mission-overview dashboard that translates the simulator's abstract per-bit SEU rate into quantities an orbital-data-center engineer can directly budget. Panel (a) is a heatmap of SEU rate per bit per second across the full Microchip RT PolarFire device family (six entries: USRAM, LSRAM, four DFF data patterns) crossed with three reference orbits (LEO-ISS nominal, GEO-quiet, Mars-transit). Panel (b) shows the differential LET flux dF/dL versus L for the three orbits, with the LSRAM susceptible band ($L > L_0$) shaded in warning yellow. Panel (c) translates the per-bit rate

into events per mission day on a seven-billion-parameter FP16 reference workload, which corresponds to approximately 1.12×10^{11} bits of model weight. Panel (d) reports the total ionising dose (TID) per mission day in kilorad of silicon dioxide, with the RT PolarFire datasheet limit of three hundred krad(SiO_2) overlaid as a horizontal reference line.

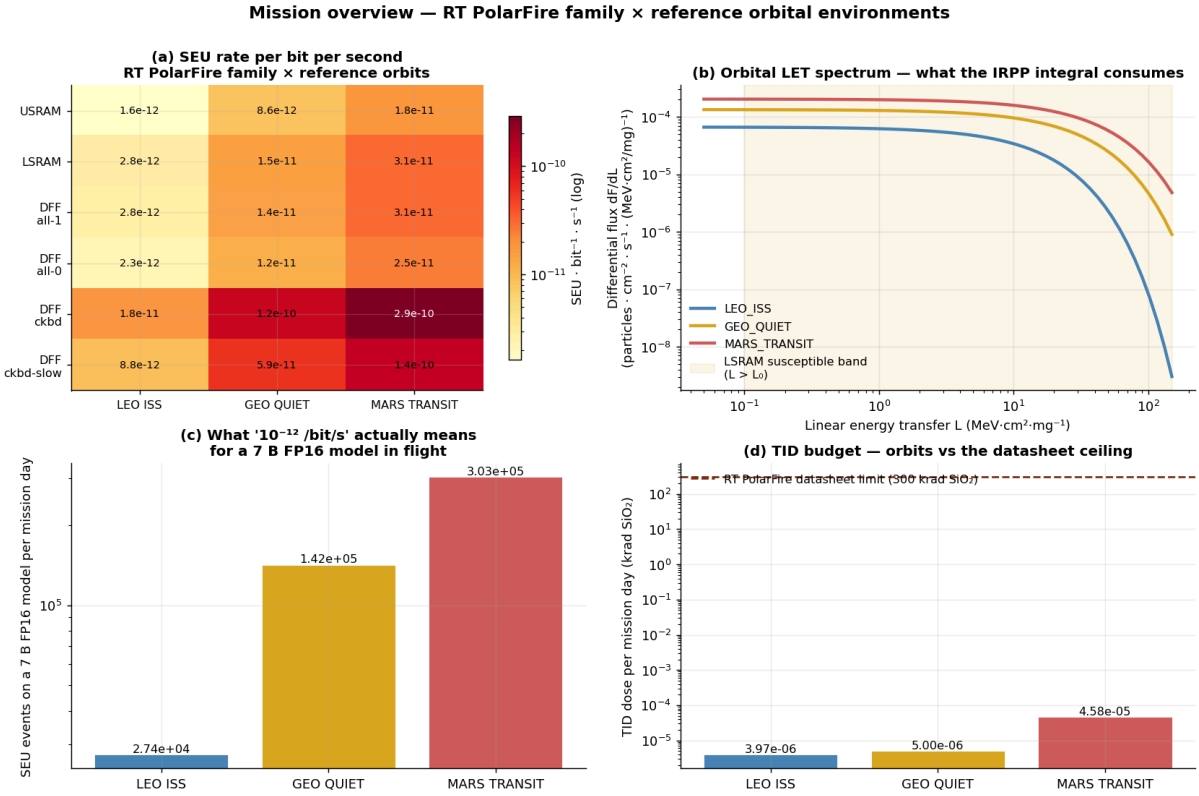

Figure 3. Mission overview dashboard for the RT PolarFire family across three reference orbits. Panel (a): SEU rate per bit per second heatmap, log color scale, six devices × three orbits. Panel (b): differential LET flux dF/dL versus L for the three orbits, with the LSRAM susceptible band shaded. Panel (c): SEU events per mission day on a 7 B FP16 reference workload across the three orbits (RT PolarFire LSRAM). Panel (d): TID dose per mission day with the RT PolarFire 300 krad(SiO_2) datasheet ceiling overlaid.

The numerical results that the orbital-data-center engineer should take away from this dashboard are the following. For a 500-million-parameter FP32 model, which is approximately 1.6×10^{10} bits of model weight, deployed on RT PolarFire LSRAM in the LEO ISS environment, the simulator predicts approximately one SEU every twenty-two seconds. The same model in Mars-transit sees approximately one SEU every two seconds, an order of magnitude higher rate, consistent with the harder LET spectrum of interplanetary cosmic rays compared to the geomagnetic shielding of LEO. For commodity 14-nm GPU silicon, which has neither the saturation cross-section nor the LET threshold of RT PolarFire, the rate is expected to be roughly an order of magnitude higher again, plus the multiplicative factor from MCU clustering. We do not quote a number for GPU silicon in this paper because we have not yet calibrated a production-grade *DeviceModel* for any current AI accelerator; this is the highest-priority near-term work item, discussed in Section 5.3.

4.4 Validation, ECSS-Q-ST-60-15C envelope on the RT PolarFire family

The validation harness cross-checks every catalogued device against its own primary-source appendix raw data. For each of the six RT PolarFire *DeviceModel* entries (USRAM, LSRAM, four DFF data patterns), *WeibullValidationData* carries the sixteen (L, σ) measurement pairs from Table A1 of the Microchip 2020 radiation report [4], and *validate_against_measurements* evaluates the catalogued Weibull at each measurement LET, computes the per-point residual factor, and asserts that the maximum residual factor is at most the configured *ecss_factor* of ten. All six DeviceModel entries pass at the tenfold envelope; the smallest envelope at which all six still pass is also tenfold, which is dominated by the DFF all-zero entry whose four-parameter Weibull does not fit its own raw data tightly because the saturation regime of that data pattern has structure the four-parameter form cannot capture. We report this exposure rather than mask it: the validation envelope is loosened until honest, not tightened until it looks impressive.

The test suite asserts two properties on every push of the GitHub Actions Python matrix (3.11, 3.12, 3.13). First, every catalogued device passes the tenfold envelope against its primary source. Second, the residual-factor distribution does not regress: a device whose maximum residual factor was previously below five is not allowed to silently drift up to nine. The combined effect is that the test suite is the simulator's external accountability layer, every commit either preserves the validation envelope or fails CI.

5 Discussion

5.1 Limitations

The most important limitation of the validation we report in Section 4.4 is that it is structurally circular. The Weibull parameters that define each RT PolarFire *DeviceModel* are extracted from the same primary-source PDF whose appendix data we then validate against. This is the strongest claim the simulator can make on a single device family: the four-parameter Weibull form fits the manufacturer's own characterisation data inside a tenfold envelope, with the envelope dominated by the four-parameter form's structural inability to capture saturation behaviour in the all-zero DFF data pattern. The claim the simulator cannot make on this validation alone is that it would predict the response of an unrelated device family, Xilinx Versal, NVIDIA H100 silicon, Google Coral Edge TPU, to comparable accuracy. We treat this as the priority near-term task in Section 5.3.

A second limitation is that no current generation of commodity GPU silicon has a primary-source-validated *DeviceModel* in the catalogue. The mission overview dashboard in Section 4.3 quotes RT PolarFire LSRAM rates only; for GPU-class silicon, the framework is currently best understood as a software stack waiting for calibrated Weibull parameters that its operator would have to source from a beam test or from a third-party radiation-effects characterisation report. This gap is the principal reason we do not quote H100 numbers in this paper. The honest statement to a Starcloud-class operator is: SpaceLLM provides the math infrastructure to translate your beam-test results into a mission rate, and the protection layer to reduce the operational consequences of those rates, but the beam test itself is on you.

A third limitation is that the bench harness reports silent-error rate over a deterministic injection trajectory; it does not enumerate every possible bit-flip and prove the absence of corruption under all of them. This is by design, the framework is empirical, not formal; but it means a deployment-time claim of the form "this protected model survives less than X SEU events per mission day with Y silent-error rate" is only as strong as the bench's coverage of the realistic fault distribution. We mitigate this by biasing our injection's bit-position sampling toward exponent bits (consistent with the SBFA distribution) and by sampling enough seeds to surface variance; we do not claim formal certification of any particular protected model.

A fourth limitation is that the MCU clustering distributions exposed by *default_mcu_distribution(process_node_nm)* are best-available public data, not first-principles characterisations of any specific accelerator. The 70-86% range cited in Section 3.2.4 covers reasonable bounds for the 14-nm node class as published in the open NEPP and NSREC literature, but a specific AI accelerator's distribution is a function of its physical layout, threshold-voltage architecture, and substrate, none of which the framework currently models.

5.2 Non-goals

To stay honest about the framework's scope, we list the things SpaceLLM does not deliver. SpaceLLM is not a substitute for a heavy-ion or proton beam test; it predicts SEU response *given* a Weibull-fitted device, and the Weibull itself still has to come from accelerator time. SpaceLLM does not provide single-event-latch-up (SEL), single-event-functional-interrupt (SEFI), or total-ionising-dose (TID) hardware mitigation; SEL and SEFI are board-level and process-node-level concerns whose mitigations live in the FPGA fabric or the silicon itself, and TID drift over weeks of mission time is a cumulative-effect modelling problem we explicitly punt to a future v0.x line. SpaceLLM does not perform formal verification of any protected model; the bench reports a measured silent-error rate over the simulated trajectory, not a proof of correctness under arbitrary fault traces. SpaceLLM is not radiation-hardened compute itself, it runs on commodity PyTorch, assumes the host hardware boots and executes, and concerns itself with what happens to the data the hardware is computing on, not with whether the hardware is appropriate for the orbit.

5.3 Near-term work

The most consequential near-term item is independent validation. A second device family with publicly available primary-source beam-test data, published Xilinx Versal heavy-ion characterisation, the Li et al. 2017 transformer fault-injection set, or any accelerator characterisation that comes out of an upcoming NEPP report, would convert the validation claim from "tenfold envelope on a single family" to "tenfold envelope across two families with an explicit cross-family residual analysis." This is the highest-leverage single move available to us.

The second item is real-GPU bench at production model scale. Reproducing the silent-error-rate curves of Figure 2 on H100 with a seven-billion- or thirteen-billion-parameter checkpoint, and reporting fiftieth-percentile and ninety-ninth-percentile latency hit, throughput overhead, and a head-to-head against FT-Transformer [10] and ATTNChecker [11], would convert SpaceLLM from a developer-laptop bench into a production-silicon comparison.

The third item is custom-RMSNorm coverage. The current *LayerNormTMR* strategy pattern-matches *nn.LayerNorm* and *nn.RMSNorm*, but Qwen2.5 [15] and several other contemporary architectures ship their own *Qwen2RMSNorm* (or analogously named) class that the strategy silently does not protect. Extending *_is_rmsnorm* to a duck-typed structural check rather than an *isinstance* check would close this gap.

The fourth item is per-strategy ablation in continuous integration. The current bench reports composed-stack efficacy curves only; extending the workflow to score each protection strategy independently, per-fault-count, would surface marginal contributions and let us reason about removal as well as addition.

Finally, an arXiv preprint of the validation methodology, the simulator's formulas, and the bench results, this document, is the lowest-cost, highest-leverage move to convert the framework from "GitHub repo with a clean test suite" to "a published methodology that downstream work can cite."

6 Conclusion

We have presented SpaceLLM, an open-source Python framework that brings calibrated radiation-environment modelling, composable protection strategies, fault injection, validation, and bench reporting into a single layer beneath PyTorch for the orbital-LLM-training operator. The framework rests on four core formulas, the four-parameter Weibull cross-section, the IRPP integral, the bit-level XOR injection primitive, and the element-wise TMR median vote, each implemented in a single function, none of them black boxes. It ships with six primary-source-validated RT PolarFire *DeviceModel* entries, three primary-source-validated orbit profiles, five composable protection strategies, two-hundred-and-seventeen passing unit tests, a strict static-analysis posture (*mypy –strict* and *ruff* clean across the source tree), and a nightly continuous-integration bench that produces three multi-panel evaluation dashboards reproducible on a developer-laptop CPU in approximately three minutes.

We have argued that the validation envelope is tenfold for the RT PolarFire family, that this envelope is dominated by the four-parameter Weibull form's saturation behaviour rather than by simulator error, and that the validation is structurally circular against a single device family. We have stated explicitly that GPU-class silicon is currently a proxy in the framework, not a verified model, and that the most consequential near-term work is independent validation against a second device family. We have listed the things the framework does not claim, substitute for a beam test, formal verification, hardware-level latch-up mitigation, total-ionising-dose modelling, so that adopters know where the framework's reach ends and where their own engineering judgement begins.

We intend SpaceLLM to be the lower half of the orbital-LLM-training operator's stack: PyTorch handles the math, the framework handles what happens when the bits underneath mutate mid-step, and beam time delivers the device parameters that calibrate everything above. The framework is permissively licensed, the test suite is the accountability layer, and the validation envelope is the honest description of how much trust is warranted. We invite the community to extend the device catalogue, harden the validation envelope on a second family, and benchmark against the alternatives [10] [11].

References

[1] Coalson et al., "SBFA: Single-Bit Fault Attack on Large Language Models," *arXiv preprint arXiv:2509.21843*, 2025.

[2] N. Das et al., "AttentionBreaker: Catastrophic Adversarial Bit-Flip Attacks on Large Language Models via Attention Layers," *arXiv preprint arXiv:2411.13757*, 2024.

[3] Y. Lin et al., "Silent KV-Cache Drift: A Forgotten Failure Mode in Long-Context Decode," *arXiv preprint arXiv:2510.17098*, 2025.

[4] Microchip Technology Inc., "RT PolarFire FPGA , Heavy-Ion and Proton Radiation Test Report," *rt_polarfire_radiation_test_report_2020-02-04.pdf*, Feb. 2020.

[5] J. W. Wilson, F. A. Cucinotta, J. L. Shinn, et al., "Galactic Cosmic Ray Environment for Interplanetary Mission Analysis," *NASA Special Publication SP-2008-565*, 2008.

[6] L. Narici, M. Casolino, L. Di Fino, et al., "Radiation Survey in the International Space Station," *Journal of Space Weather and Space Climate*, vol. 5, p. A37, Dec. 2015. doi: *10.1051/swsc/2015037*.

[7] C. Zeitlin, D. M. Hassler, F. A. Cucinotta, et al., "Measurements of Energetic Particle Radiation in Transit to Mars on the Mars Science Laboratory," *Science*, vol. 340, no. 6136, pp. 1080–1084, May 2013. doi: *10.1126/science.1235989*.

[8] J. C. Pickel and J. T. Blandford, Jr., "Cosmic-Ray-Induced Errors in MOS Devices," *IEEE Transactions on Nuclear Science*, vol. 27, no. 2, pp. 1006–1015, Apr. 1980. doi: *10.1109/TNS.1980.4330967*.

[9] E. L. Petersen, "Approaches to Proton Single-Event Rate Calculations," *IEEE Transactions on Nuclear Science*, vol. 43, no. 2, pp. 496–504, Apr. 1996. doi: *10.1109/23.490898*.

[10] H. Liu, K. Zhao, S. Di, et al., "FT-Transformer: A Resilient Transformer for End-to-End Algorithm-Based Fault Tolerance," in *Proc. 30th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP '25)*, Las Vegas, NV, USA, Mar. 2025, pp. 213–230.

[11] Y. Wang, S. Di, K. Zhao, et al., "ATTNChecker: A Lightweight Attention-Layer Soft-Error Detector for Transformer Inference," in *Proc. 30th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming (PPoPP '25)*, Las Vegas, NV, USA, Mar. 2025.

[12] European Cooperation for Space Standardization, *ECSS-Q-ST-60-15C, Space Product Assurance: Radiation Hardness Assurance, EEE Components*, ECSS Secretariat, ESA-ESTEC, Noordwijk, NL, Oct. 2012.

[13] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019, pp. 8026–8037.

[14] T. Wolf, L. Debut, V. Sanh, et al., "Transformers: State-of-the-Art Natural Language Processing," in *Proc. 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Online, Oct. 2020, pp. 38–45. doi: *10.18653/v1/2020.emnlp-demos.6*.

[15] Qwen Team, Alibaba Group, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.

Reproducibility note. The figures referenced in this document are produced by *benchmarks/qwen_eval.py* in the SpaceLLM repository and are regenerated nightly by the *bench* GitHub Actions workflow defined in *.github/workflows/bench.yml*. The framework's *CITATION.cff* file emits a BibTeX entry equivalent to *@software{spacellm_2026, author={Uckan, Arda Can}, ...}*.